Legal SME for Civil Law (India): Contracts and Agreements

Bitter Lesson - Akshit Kumar, Hardik Mittal

Domain Definition & SME Design

Legal expert systems in the 80s–90s failed

- they tried to encode statutes directly as rules
- assumption: the text itself contained the operative logic
- real “fundamental principles” of law live in judicial reasoning, not in the bare wording of Acts

Our Design

- instead of hard-coding legal rules or workflows
- Expose the agent to a large corpus
- Give it appropriate and robust tools
    - Let it search document metadata and chunks with page information
    - Let it “zoom out” by giving it tools which let it access pages directly

The scaling principle behind this philosophy is discussed in Sutton’s classic piece: The Bitter Lesson

To support this, we employ a comprehensive data annotation and indexing process.

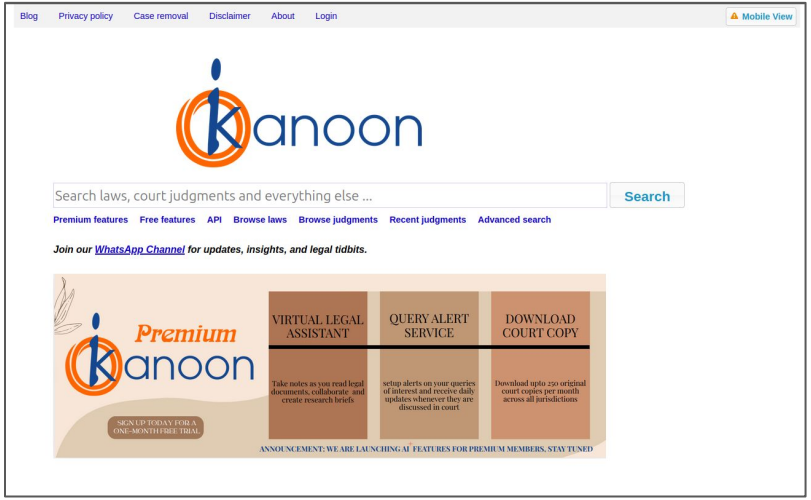
> “Behind the Acts is a **general body of common law** and equity applying to all companies irrespective of their nature, and it is there that most of the fundamental principles will be found. The fact that **most of the fundamental principles will not be found in the statutes** themselves, but in the **reported decisions of cases**, has immense implications for those who would work outwards from the detailed wording of individual statutes.”
>
> ~ Legal Knowledge Based Systems: JURIX 92 — Information Technology and Law

Data Collection & Organisation

- https://indiankanoon.org/
  - Scraped 15k+ court documents from the website

- Got all the related Acts and Sections from the constitution related to Contracts and Agreements.

- OCR on the pdf documents using DeepSeek-OCR to convert to markdown



Metadata Creation

- Ask `google/gemma-3-4b-it` to create metadata
- Rationale
  - Instead of just searching over chunks, we can now do text search using BM25 over the documents
  - This is especially useful for a legal case scenario where we need to look for specific cases, sections, courts etc. (where vector search might fail)

```
- "doc_id": string
- "case_title": string, if there is no explicit title,
create an appropriate title
- "doc_type": one of ["judgment", "contract", "statute",
"paper", "other"]
- "decision_date": string, DD/MM/YY or null
- "decision_date_human": string, DD Month YYYY or null
- "statutes_mentioned": List[string], can be empty
- "jurisdiction": string or null
- "court": string
- "parties": list of objects with keys:
    - "name": string
    - "role": string, such as "plaintiff", "defendant",
"petitioner", "respondent", "author", "other"
- "topics": list of 3 to 5 short strings (1 to 5 words
each)
- "summary": at most 2 sentences summarising the whole
document
- "sections": list of at most 5 objects with keys:
    - "heading": short string
    - "summary": at most 2 sentences summary of that
section
    - "pages": A two-element JSON list of integers in
the form [start_page, end_page].
```

Preprocessing and Chunking

- Cleaning of the DeepSeek OCR generated markdown
  - Basic cleaning using regex like removing extra white spaces, blank lines (was not required much since court documents are structured and not noisy)
  - Scrub away DeepSeek OCR tags (reference, detection tags), strange punctuation, empty table cells.

- Chunking
  - Find all the text in a page for all the documents and add this page info in the chunk
  - Overlapping chunks of variable sizes (384, 256, 128) - each with page metadata attached.

CHUNKS LOOK LIKE THIS

```
{
  384: [
    {
      "chunk_id": 0,
      "granularity": 384,
      "text": "...",
      "pages": [1]
    },
    {
      "chunk_id": 1,
      "granularity": 384,
      "text": "...",
      "pages": [1,2]
    },
    ...
  ],
  256: [...],
  128: [...]
}
```

Embeddings and Indexing

We employ both keyword based and vector based retrieval systems, letting the agent decide which one to use based on the situation.

- **FAISS for semantic search**
  - Encodes the query into an embedding vector, normalizes it, and queries FAISS.
  - Returns the (`score, idx`)
- **BM25 for keyword-based search**
  - Over a concatenated string of all the previous extracted metadata
  - Finds the most related metadata to the query and returns it corresponding `doc_id, score, md_path, pdf_path, metadata`
- **SQLite for persistent metadata storage**
  - **`docs`**: one row per document; stores:
    - `doc_id`: unique identifier.
    - `md_path`: Markdown path.
    - `pdf_path`: original PDF path.
    - `extracted_meta_json`: JSON of structured metadata (from your previous model).
    - `raw_output`: raw model output if metadata parse failed.
  - **`chunks`**: one row per text chunk; stores:
    - `vector_id`: corresponds to FAISS index entry.
    - `chunk_uid`, `doc_id`, `md_path`.
    - `pages_json`: JSON array of page numbers.
    - `granularity`: token window size.
    - `chunk_text`: actual chunk.

```
@dataclass
class ChunkRecord:
    vector_id: int  # index in FAISS
    chunk_uid: str
    doc_id: str
    md_path: str
    pages: List[int]
    granularity: int
    text: str
```

```
self.bm25_corpus_tokens: List[List[str]]
self.bm25_doc_ids: List[str]
self.bm25: # bm25 scoring logic
```

- We also create two more accompanying retrieval helper functions
  - **get_doc_metadata(doc_id)**
  - **get_doc_content(doc_id, pages:List[int])**

This lets the agent "*zoom out*", when required, to get more context on the chunks and docs it finds

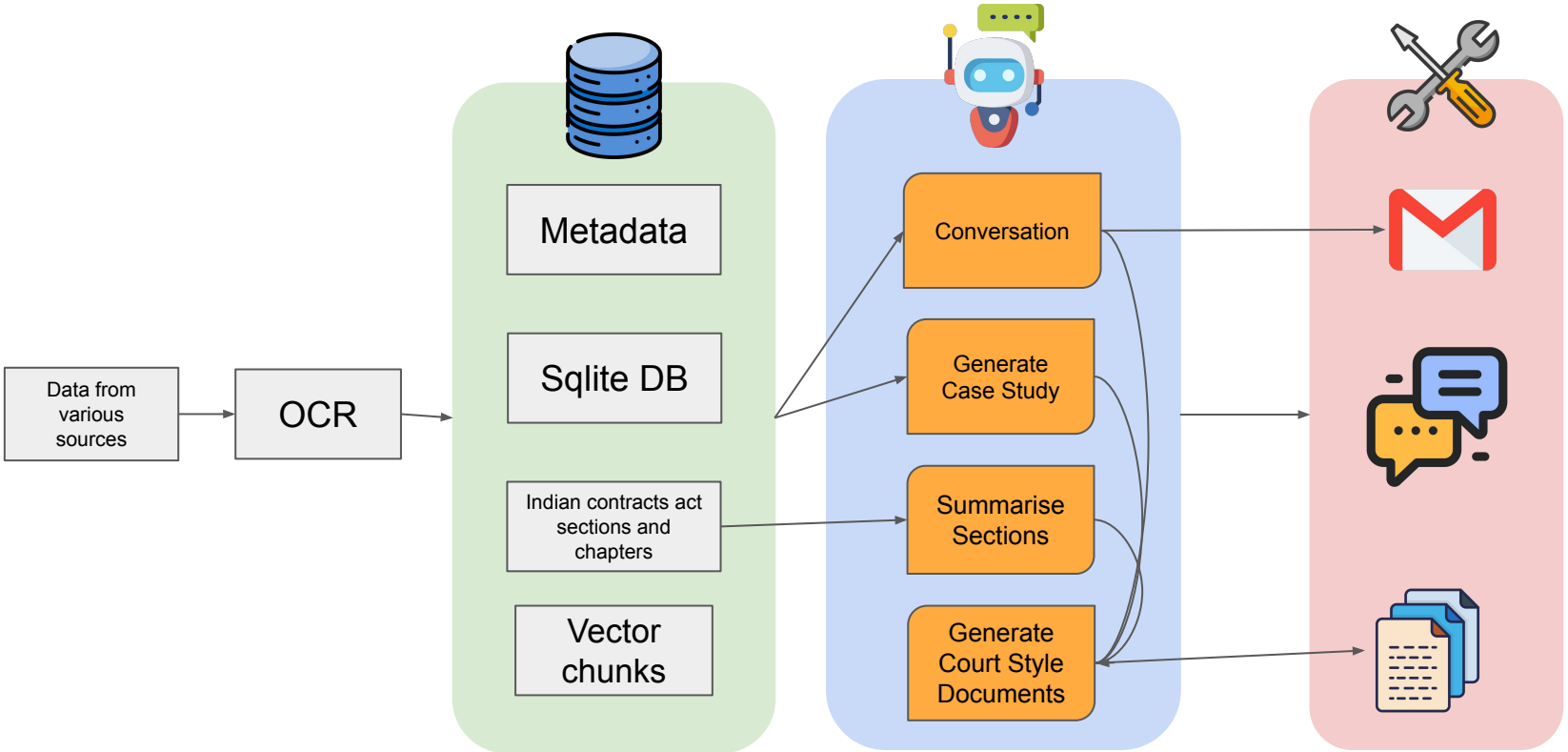
This is a clear alternative to other strategies like providing chunks before and after the relevant chunks, except we believe this is much more scalable since the agent can decide by itself how big the context around the chunk it really needs.

Agent Workflows, Subagents & Tools

- Meta Agent which chooses which sub agent to call
  - **Generate Case Study**
    - find relevant doc with reranking
    - generate case study
  - **Summarise Statutes**
    - Has access to Indian Contracts Act and can search in that specifically (aside the main db)
    - summarises
  - **Generate Court-Style Document**
    - Creates any kind of document using pandoc
    - Markdown Repair Agent
  - **Generic Answer**
    - search_faiss
    - search_bm25
    - get_doc_metadata
    - get_doc_content
    - serve_pdf link
    - email_text (with attachments)
    - render_markdown

Prompting Strategies

- Exact workflow on how to use different tools for various use cases as defined above

- Specifying the exact way to add citations along with correct links from the back end server

System Architecture and Web App